Smart Transport Tunis — Étapes 5 et 6

Étape 5 — Générer et sélectionner des idées

La solution retenue est **Smart Transport Tunis**, une application web responsive qui combine suivi GPS des bus, estimation du temps d'arrivée (ETA), niveau de saturation, signalement des perturbations et recherche d'alternatives. Elle obtient 21/25, le meilleur score parmi six idées comparées.

5.1 Rappel du problème

Les usagers des transports publics du Grand Tunis ne savent pas où se trouve leur bus, combien de temps ils doivent attendre, si le bus est saturé ou en panne, ni quelle alternative prendre.

5.2 Idées de solutions générées

Six approches différentes ont été imaginées pour résoudre le problème ; le tableau en présente le principe.

N°	Idée	Principe
A	Plateforme de suivi GPS + ETA + affluence + signalements + alternatives	Carte interactive avec position des bus, temps d'arrivée estimé, niveau de saturation et alertes
B	Application d'horaires et de fiches lignes	Consultation statique des lignes, arrêts et horaires planifiés
C	Application 100 % collaborative (type « Waze du bus »)	Les voyageurs signalent position, retards, pannes et affluence, sans GPS véhicule
D	Capteurs IoT embarqués (comptage de passagers + GPS dédié)	Boîtiers installés dans les bus pour mesurer précisément le remplissage et la position
E	Service d'information par SMS / WhatsApp / chatbot	L'usager envoie un numéro de ligne ou d'arrêt et reçoit une estimation
F	QR codes et affichage aux arrêts	Panneaux ou QR codes donnant les prochains passages

5.3 Critères de sélection et notation

Les critères sont ceux de l'énoncé : valeur apportée, originalité, faisabilité, ressources disponibles et temps de réalisation. Chaque idée est notée de 1 (faible) à 5 (excellent). Pour « Ressources » et « Temps », 5 signifie « peu de ressources nécessaires » et « rapide à réaliser ».

Idée	Valeur	Originalité	Faisabilité	Ressources	Temps	Total /25
A. Suivi GPS + ETA + affluence + alertes	5	4	4	4	4	**21**
C. Application collaborative	3	2	5	5	4	19
B. Horaires statiques	2	1	5	5	5	18
E. SMS / WhatsApp	3	3	3	4	3	16
F. QR codes / affichage aux arrêts	2	3	3	3	3	14
D. Capteurs IoT embarqués	4	5	1	1	1	12

5.4 Analyse des idées écartées

Chaque idée écartée l'est pour une raison précise, résumée ci-dessous.

- **B (horaires statiques)** : facile à réaliser, mais elle ne résout pas l'incertitude de l'attente.
- **C (application collaborative)** : réalisable, mais peu fiable seule, car la qualité dépend du nombre de contributeurs. Elle est conservée comme mécanisme complémentaire (signalements et affluence).
- **D (capteurs IoT)** : la plus originale, mais elle exige du matériel, des partenariats et un budget.
- **E et F (SMS et QR codes)** : elles répondent partiellement au problème et restent des extensions possibles.

5.5 Solution retenue

Smart Transport Tunis est une application web responsive qui combine suivi GPS des bus, estimation du temps d'arrivée (ETA), niveau de saturation déclaré par les voyageurs, signalement des perturbations et recherche d'alternatives.

5.6 Justification du choix

Le tableau confronte la solution retenue aux cinq critères de l'énoncé.

Critère	Justification
Valeur apportée	Elle traite les 5 problèmes identifiés (saturation, faible fréquence, attente incertaine, pannes, absence d'alternatives) dans une seule application.
Originalité	Elle combine données GPS, contributions des voyageurs et alternatives. Elle affiche aussi la fiabilité et la fraîcheur de chaque information.
Faisabilité	Les technologies sont libres et connues (React, Leaflet/OpenStreetMap, FastAPI, PostgreSQL). Un smartphone ou un simulateur peut remplacer le GPS d'un bus pour le prototype.
Ressources disponibles	Aucun matériel coûteux : un smartphone, des outils open source et un hébergement local ou cloud gratuit suffisent.
Temps de réalisation	Le projet est découpable en modules indépendants (GPS, ETA, signalements, interface), que les 5 membres de l'équipe développent en parallèle. Les outils choisis (React, FastAPI, PostgreSQL) sont déjà maîtrisés, ce qui évite une longue phase d'apprentissage et permet d'obtenir une solution fonctionnelle dans le temps imparti.

Étape 6 — Concevoir la solution

Smart Transport Tunis repose sur une architecture client-serveur : une application React pour les voyageurs, une API FastAPI qui regroupe cinq modules métier, et une base PostgreSQL. Le livrable comprend le fonctionnement général (6.1), le schéma fonctionnel (6.2), les composantes (6.3), les interactions avec l'utilisateur (6.4), les ressources et technologies (6.5) et la description détaillée des fonctionnalités (6.6).

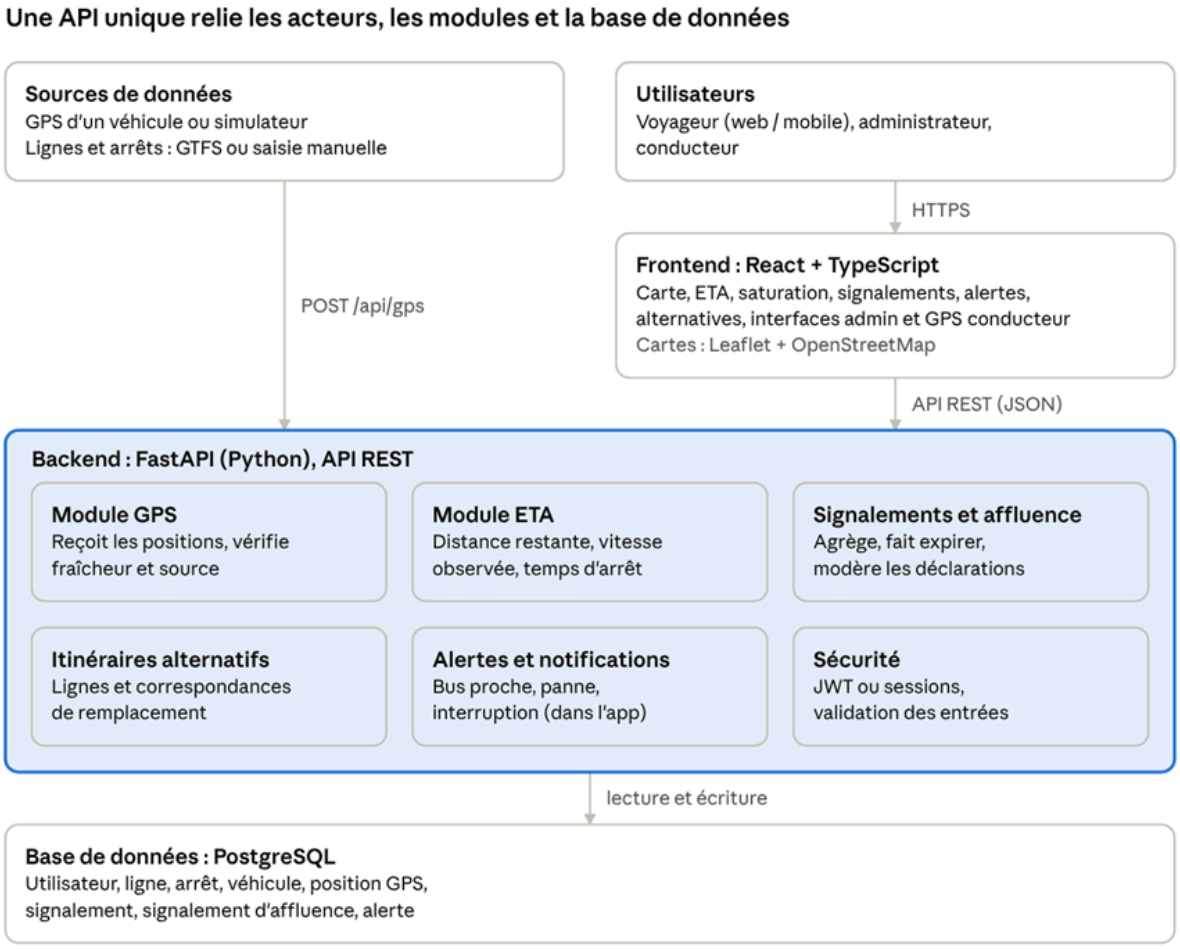
6.1 Fonctionnement général

Le fonctionnement suit six étapes, de l'envoi de la position du bus jusqu'à l'affichage de l'information au voyageur.

1. Un véhicule (smartphone ou simulateur) envoie périodiquement sa position GPS au serveur.
2. Le serveur enregistre la position et calcule l'ETA pour les arrêts suivants.
3. Le voyageur ouvre l'application, choisit une ligne ou un arrêt et voit sur la carte les bus, l'ETA et le niveau de saturation.
4. Le voyageur peut déclarer l'affluence ou signaler un incident. Le serveur agrège ces signalements, les modère et les fait expirer.
5. En cas de panne, de retard important ou de saturation, le système affiche une alerte et propose un trajet alternatif.
6. Si les données GPS sont trop anciennes, l'application affiche « estimation indisponible » et non une heure fictive.

6.2 Schéma fonctionnel

Le schéma montre les quatre couches de la solution : sources de données et utilisateurs, frontend, backend et base de données.



Les voyageurs passent par le frontend, les positions GPS arrivent directement à l'API (POST /api/gps), et les modules du backend lisent et écrivent dans PostgreSQL.

6.3 Principales composantes

Chaque composante a un rôle précis dans la solution.

Composante	Rôle
Application voyageur	Recherche, carte, ETA, saturation, signalements, alertes, alternatives
Interface administrateur	Gestion des lignes, arrêts et véhicules, modération, supervision

Interface GPS	Page web simple utilisée sur le smartphone du conducteur : elle permet de se connecter, de démarrer ou d'arrêter l'envoi de la position GPS, de vérifier l'état de la connexion et de signaler une panne.
API Backend	Point d'entrée unique : sécurité, validation et logique métier
Module GPS	Reçoit les positions, vérifie leur fraîcheur et leur source
Module ETA	Calcule l'heure d'arrivée estimée
Module signalements et affluence	Agrège, fait expirer et modère les déclarations
Module itinéraires alternatifs	Cherche des lignes ou correspondances de remplacement
Module alertes	Génère les messages (bus proche, panne, interruption)
Base de données	Stocke lignes, arrêts, positions, signalements, alertes

6.4 Interactions avec l'utilisateur

Trois types d'acteurs utilisent la solution, chacun avec ses propres actions.

Acteur	Actions principales
Voyageur	Rechercher une ligne, voir les bus sur la carte, consulter l'ETA, déclarer l'affluence (peu rempli, moyennement rempli, saturé), signaler un incident, recevoir des alertes, chercher une alternative
Conducteur	Se connecter, démarrer ou arrêter l'envoi GPS, voir l'état de connexion, signaler une panne
Administrateur	Gérer lignes et arrêts, modérer les signalements, suivre les véhicules, consulter les statistiques

6.5 Ressources et technologies

La solution repose sur des outils libres et connus.

Catégorie	Éléments
Frontend	React + TypeScript, Tailwind CSS
Cartographie	Leaflet + OpenStreetMap
Backend	FastAPI (Python), API REST avec actualisation périodique
Base de données	PostgreSQL
Données réseau	GTFS si disponible, sinon saisie manuelle de quelques lignes
GPS	Smartphone ou simulateur
Sécurité	HTTPS, JWT ou sessions, validation des entrées
Outils	Git + GitHub, Pytest
Déploiement	Local ou cloud pour la démonstration
Ressources humaines	5 étudiants
Données de test	Lignes, arrêts et parcours, positions GPS simulées, signalements et affluences de test

6.6 Description détaillée des fonctionnalités

Cette section décrit chaque fonction de la solution selon la même structure : son objectif, son fonctionnement et son cas particulier.

6.6.1 Consultation des lignes et des arrêts

- **Objectif :** permettre au voyageur de trouver sa ligne et de connaître son parcours.
- **Fonctionnement :** le voyageur recherche une ligne par son numéro ou son nom. L'application affiche la direction, la liste des arrêts desservis, le parcours sur la carte et les horaires planifiés lorsqu'ils existent.
- **Cas particulier :** si les horaires planifiés ne sont pas disponibles pour une ligne, seuls le parcours et les arrêts sont affichés.

6.6.2 Suivi GPS des bus

- **Objectif :** montrer au voyageur où se trouve le bus en ce moment.
- **Fonctionnement :** le véhicule envoie régulièrement sa position au serveur. La carte affiche le bus avec son numéro de ligne, sa direction et l'heure de sa dernière position, et se met à jour automatiquement.

- **Cas particulier :** chaque position est étiquetée « réelle », « simulée » ou « indisponible », pour que le voyageur sache toujours d'où vient l'information.

6.6.3 Estimation du temps d'arrivée (ETA)

- **Objectif :** indiquer combien de temps il reste avant l'arrivée du bus à un arrêt.
- **Fonctionnement :** le voyageur choisit une ligne et un arrêt. L'application affiche le prochain bus suivi et son heure d'arrivée estimée, calculée à partir de ses dernières positions GPS. L'estimation est actualisée à chaque nouvelle position reçue.
- **Cas particulier :** si les données GPS sont trop anciennes, l'application affiche « estimation indisponible » et jamais une heure inventée.

6.6.4 Niveau de saturation des bus

- **Objectif :** informer le voyageur de l'affluence dans le bus avant qu'il monte.
- **Fonctionnement :** les voyageurs déclarent le remplissage selon trois niveaux : peu rempli, moyennement rempli ou saturé. Le serveur regroupe les déclarations récentes et affiche le niveau estimé avec son heure de mise à jour.
- **Cas particulier :** si les déclarations sont trop peu nombreuses, l'application affiche « Affluence inconnue ». Les niveaux sont des estimations des voyageurs, pas des comptages exacts.

6.6.5 Signalement des perturbations

- **Objectif :** permettre aux voyageurs de prévenir les autres lorsqu'un problème survient.
- **Fonctionnement :** le voyageur choisit un type d'incident (bus en retard, bus en panne, trajet interrompu, bus saturé ou autre), la ligne concernée et, s'il le souhaite, une description. Le signalement apparaît avec son heure de publication et son statut de vérification.
- **Cas particulier :** les signalements expirent après un certain temps et peuvent être regroupés ou modérés, afin qu'une information fausse ne reste pas affichée.

6.6.6 Alertes et notifications

- **Objectif :** prévenir le voyageur des événements importants qui concernent son trajet.
- **Fonctionnement :** l'application affiche une alerte lorsque le bus approche de l'arrêt, lorsqu'une panne est signalée sur la ligne, ou lorsqu'une interruption de service est confirmée.
- **Cas particulier :** dans l’application, les alertes sont visibles uniquement dans l'application ; les notifications sur le téléphone sont une évolution possible.

6.6.7 Recherche d'itinéraires alternatifs

- **Objectif :** aider le voyageur à trouver une autre solution lorsque son bus est en panne, très en retard ou saturé.
- **Fonctionnement :** le voyageur indique son départ et sa destination. L'application propose les lignes et correspondances disponibles, avec le temps de trajet estimé.
- **Cas particulier :** la couverture se limite aux lignes enregistrées dans le système.

6.6.8 Administration et modération

- **Objectif :** garder des informations fiables et gérer les données de la solution.
- **Fonctionnement :** l'administrateur gère les lignes, les arrêts et les véhicules. Il consulte les signalements des voyageurs et les valide, les regroupe ou les supprime, et il suit la position des véhicules autorisés.
- **Cas particulier :** l'accès à l'interface d'administration est protégé par une authentification.